Basic Factory Creation

Factory_bot provides four ways of turning a factory into a Ruby object. For the project factory we were just looking at, the four ways are as follows:

- build(:project), which returns a model instance that has not been saved to the database.
- create(:project), which returns a model instance and saves it to the database.
- attributes_for(:project), which returns a hash of all the attributes in the factory that are suitable for passing to ActiveRecord#new or ActiveRecord#create. This method is most often useful for creating a hash that will be sent as params to a controller test.
- build_stubbed(:project), which is almost magical. Like build, it returns an unsaved model object. Unlike build, it assigns a fake ActiveRecord ID to the model and stubs out database-interaction methods (like save) such that the test raises an exception if they are called.

Your existing project_spec uses basic Project and Task instances. You can rewrite it as follows:

data/01/spec/models/project_spec.rb

```
describe "without a task" do
  let(:project) { FactoryBot.build_stubbed(:project) }
  it "considers a project with no tasks to be done" do
    expect(project).to be_done
  end

  it "properly estimates a blank project" do
    expect(project.completed_velocity).to eq(0)
    expect(project.current_rate).to eq(0)
    expect(project.projected_days_remaining).to be_nan
    expect(project).not_to be_on_schedule
  end
end

describe "with a task" do
  let(:project) { FactoryBot.build_stubbed(:project, tasks: [task]) }
  let(:task) { FactoryBot.build_stubbed(:task) }

  it "knows that a project with an incomplete task is not done" do
    expect(project).not_to be_done
  end

  it "marks a project done if its tasks are done" do
    task.mark_completed
    expect(project).to be_done
  end
end
```